A NOVEL IDS FRAMEWORK COMBINING PCA AND RANDOM FOREST

A PROJECT REPORT

Submitted by

Roshan Kumar S	**RA2111030020018**
Mageshwaran SD	**RA2111030020028**
Krithik S R	**RA2111030020047**

Under the guidance of

Dr. U. Surendar, ME, Ph.D, MISTE, IEEE
Assistant Professor,
Department of Computer Science and Engineering

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

with specialization in

CYBER SECURITY

of

FACULTY OF ENGINEERING AND TECHNOLOGY

S R M INSTITUTE OF SCIENCE AND TECHNOLOGY
CHENNAI - 600089

May 2025

S R M INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **"A NOVEL IDS FRAMEWORK COMBINING PCA AND RANDOM FOREST"** is the Bonafide work of **Roshan Kumar S [RA2111030020018], Mageshwaran SD [RA2111030020028] and Krithik S R [RA2111030020047]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr. U. Surendar, M.E, PhD.,

MISTE, IEEE,

Assistant Professor

Department of Computer Science and Engineering
SRM Institute of Science and Technology, Chennai-89.

SIGNATURE

Dr. J. Shiny Duela M.E, PhD.,

Associate Professor and Head of Department

Department of Computer Science and Engineering
SRM Institute of Science and Technology, Chennai-89.

Submitted for the project viva-voce held on_____at SRM Institute of Science and Technology, Chennai -600089.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, CHENNAI - 89

DECLARATION

We hereby declare that the entire work contained in this project report titled "**A NOVEL IDS FRAMEWORK COMBINED WITH PCA AND RANDOM FOREST"** has been carried out by **ROSHAN KUMAR S [RA2111030020018], MAGESHWARAN SD [RA2111030020018] AND KRITHIK S R [RA2111030020047]** at SRM Institute of Science and Technology, Ramapuram, Chennai- 600089, under the guidance of **Dr.U.Surendar, ME, Ph.D, MISTE, IEEE, Assistant Professor,** Department of Computer Science and Engineering.

Place: Chennai

Date:

MAGESHWARAN S D

ROSHAN KUMAR S

KRITHIK S R

Own Work Declaration

Department of Computer Science and Engineering

SRM Institute of Science and Technology
Own Work Declaration form

Degree/ Course : B. Tech Computer Science and Engineering with Specialization in Cyber Security
Student Name : ROSHAN KUMAR S, MAGESHWARAN SD, KRITHIK S R
Registration Number : RA2111030020018, RA2111030020028, RA2111030020047

Title of Work : A NOVEL IDS FRAMEWORK COMBINED WITH PCA AND RANDOM FOREST

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not our own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g., fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalised in accordance with the University policies and regulations.

DECLARATION:		
We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.		
RA2111030020018	RA2111030020028	RA2111030020047

ACKNOWLEDGEMENT

We place on record our deep sense of gratitude to our lionized Chairman **Dr.R.SHIVAKUMAR** for providing us with the requisite infrastructure throughout the course. We take the opportunity to extend our hearty and sincere thanks to our Dean, **Dr. M SAKTHI GANESH., Ph.D.,** for maneuvering us into accomplishing the project. We take the privilege to extend our hearty and sincere gratitude to the Professor and Head of the Department, **Dr. J. SHINY DUELA M.E PhD.,** for his suggestions, support and encouragement towards the completion of the project with perfection. We express our hearty and sincere thanks to our guide **Dr.U.SURENDAR**,**M.E,PhD., MISTE, IEEE,** Assistant Professor, Computer Science and Engineering Department for his encouragement, consecutive criticism and constant guidance throughout this project work. Our thanks to the teaching and non-teaching staff of the Computer Science and Engineering Department of SRM Institute of Science and Technology, Ramapuram Campus, for providing necessary resources for our project. Our thanks to the teaching and non-teaching staff of the Department of Computer Science and Engineering of SRM Institute of Science and Technology, Chennai, for providing necessary resources for our project.

ABSTRACT

With the rapid evolution of cyber threats ensuring robust network security has become crucial for organizations and individuals. Intrusion Detection Systems (IDS) play a vital role in monitoring and identifying unauthorized access cyber-attacks and security breaches in computer networks. Traditional ideas approach often struggle with high-dimensional datasets leading to increased computational complexity and reduced efficiency. To address this, various Machine Learning (ML) techniques have been explored to improve intrusion detection accuracy. This project proposes a novel IDS framework that integrates Principal Component Analysis (PCA) with the Random Forest classification algorithm to enhance the accuracy and efficiency of intrusion detection. PCA is employed to reduce the dimensionality of the dataset by extracting the most relevant features, and thereby improving processing speed and minimizing redundant data. The random forest algorithm known for its robustness and high accuracy in classification task is then applied to effectively detect different types of attacks. By combining these techniques, the system can identify anomalies and malicious activities with greater precision while maintaining computational efficiency.

The proposed approach has been evaluated using benchmark intrusion detection datasets and real-world network traffic traces. Comparative analysis with traditional methods including Support Vector Machines (SVM), Naïve Bayes, and decision trees demonstrates that the PCA-Random Forest model achieves higher reduction rates and lower positive rates. Additionally, the ensemble learning capability of random forest enhances classification accuracy by aggregating predictions from multiple decision trees. This research highlights the effectiveness of integrating dimensionality reduction with ensemble learning to optimize IDS performance. The results indicate that this hybrid approach offers a more scalable, efficient and accurate intrusion detection mechanism compared to conventional methods. With its ability to deduct intrusions in real-time while reducing computational overhead, the proposed system presents a promising advancement in cyber security and network defense.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

IDA	INTRUSION DETECTION SYSTEMS
PCA	CPRINCIPAL COMPONENET ANALYSIS
CNN	CONVOLUTIONAL NEURAL NETWORK
ANN	ARTIFICIAL NEURAL NETWORK
RFA	RANDOM FOREST ALGORITHM

CHAPTER 1

INTRODUCTION

An Intrusion Detection System (IDS) monitors network traffic to deduct unauthorized access, cyberattacks, and malware infections. Traditional IDS models struggle with large, high dimensional datasets leading to inefficiencies in real time detection. Principal Component Analysis (PCA) reduces dimensionality by transforming high dimensional data into a lower dimensional space, eliminating irrelevant features and improving computational efficiency. Meanwhile random forest, an ensemble learning algorithm builds multiple decision trees to classify network traffic as normal or malicious, enhancing accuracy and robustness. Integrating PCA with random forest allows IDS to efficiently process large-scale cybersecurity datasets reducing false positives and improving intrusion detection. This combination ensures real time adaptability and strengthens threat detection making ideas more effective in modern network security. The approach not only enhances deduction accuracy but also optimizes performance for real time applications.

1.1 PROBLEM STATEMENT

Traditional intrusion detection systems struggle with outdated detection methods, large-scale data processing and high false positive rates, making them inefficient against evolving cyber threats. The increasing complexity of network traffic generates high dimensional data, making real-time analysis computationally expensive. To overcome these challenges, this project proposes an advanced IDS framework integrating Principal Component Analysis (PCA) and Random Forest for improved detection accuracy and efficiency. PCA reduces data dimensionality by selecting significant features, minimizing redundancy and accelerating model training. The random forest approach enhances classification performance by aggregating multiple decision trees ensuring resilience against overfitting. By combining these techniques, the IDS framework delivers a scalable, high-performance system that identifies cyber threats with great precision. This approach significantly reduces false positives making real-time intrusion reduction more reliable and effective.

1.2 AIM OF THE PROJECT

- To develop a robust Intrusion Detection System (IDS) by integrating Principal Component Analysis (PCA) and Random Forest Algorithm (RFA) for efficient threat detection.
- To reduce dimensionality of network traffic data using PCA, enhancing processing speed and eliminating irrelevant features.
- To leverage Random Forest for accurate classification of network activities into normal or intrusive, ensuring high detection rates.
- To design a user-friendly web interface that allows seamless interaction, real-time monitoring, and clear visualization of predictions.
- To improve cybersecurity measures by identifying and categorizing different attack types, aiding in proactive threat mitigation.

1.3 PROJECT DOMAIN

This project falls under the Cyber Security (CS) and Intrusion Detection Systems (IDS) domain, focusing on network anomaly detection and threat classification using Machine Learning (ML) techniques.

1.4 SCOPE OF THE PROJECT

This scope of this project is to develop an efficient and scalable Intrusion Detection System (IDS) by integrating PCA for dimensionality reduction and Random Forest for accurate classification. PCA eliminates irrelevant features, improving computational efficiency, while Random Forest enhances detection accuracy through ensemble learning. The approach focuses on minimizing false positives, ensuring reliable and precise intrusion detection.

1.5 METHODOLOGY

The proposed methodology integrates Principal Component Analysis (PCA) with the Random Forest Algorithm (RFA) to enhance the performance of Intrusion Detection Systems (IDS). The process begins with data preprocessing, where raw network traffic data is cleaned and transformed to ensure consistency. Next, dimensionality reduction is performed using PCA, which eliminates redundant and irrelevant features, retaining only the most informative attributes for classification. This step significantly reduces computational complexity while preserving essential patterns for anomaly detection. Once the feature set is optimized, the Random Forest classifier is employed to detect intrusions by building multiple decision trees and aggregating their outputs for improved accuracy and robustness. The ensemble learning approach of RFA enhances classification performance and mitigates the risk of overfitting by ensuring stability across diverse datasets. The hybrid IDS model thus effectively differentiates between normal and malicious activities, improving detection rates and reducing false positives. To further ensure scalability and real-time application, the system is tested on large datasets, assessing its ability to handle evolving cyber threats efficiently. The proposed approach is also evaluated across various cybersecurity domains such as malware classification and fraud detection, demonstrating its versatility. By leveraging PCA for feature extraction and RFA for classification, the methodology provides a resilient, adaptive, and scalable framework for securing modern networks against sophisticated cyber threats.

1.6 ORGANIZATION OF THE REPORT

Chapter 2 provides an in-depth analysis of previous research related to Intrusion Detection Systems (IDS), machine learning approaches, PCA, and Random Forest algorithms. It examines various methodologies, highlighting their strengths and limitations. The review helps establish a knowledge base, identifying research gaps and justifying the need for the proposed framework.

Chapter 3 outlines the objectives, scope, and significance of the project. It describes the problem statement, the motivation behind choosing PCA and Random Forest for IDS, and the expected outcomes. Additionally, it explains the key components and architectural design of the proposed system.

Chapter 4 details the proposed methodology integrating PCA for dimensionality reduction and Random Forest for classification. It explains how PCA helps in feature selection and how Random Forest enhances classification accuracy. The workflow, system architecture, and data processing steps are elaborated, ensuring clarity in how the system functions.

Chapter 5 presents the practical implementation of the project, including the development of the frontend, backend, and machine learning model. It also covers the datasets used, preprocessing techniques, training process, and evaluation metrics. Testing strategies, including functional and performance testing, are discussed to validate the system's accuracy and efficiency.

Chapter 6 showcases the experimental results, comparing the model's performance with existing IDS approaches. It includes performance metrics such as accuracy, precision, recall, and F1-score to assess the effectiveness of the proposed framework. Additionally, it discusses the strengths and limitations of the system based on the observed outcomes.

Chapter 7 summarizes the research findings and the contributions of the proposed IDS framework. It highlights key takeaways, such as improved accuracy, reduced false positives, and enhanced efficiency. The chapter also discusses potential future enhancements, including integration with deep learning models, real-time deployment, and adaptation to evolving cyber threats.provides the conclusion and outlines future enhancements. The proposed approach involves an end-to-end fully convolutional network for modeling the mapping between face photos and sketches. Experimental results demonstrate the efficacy of this approach. Future enhancements include refining the loss function, experimenting with different databases, and exploring correlations with non-photorealistic rendering methodologies.

Chapter 8 contains the source code and details about the poster presentation. It includes sample code snippets for reference.

CHAPTER 2

LITERATURE REVIEW

The Literature Review explores existing Intrusion Detection Systems (IDS), focusing on machine learning-based approaches, feature selection techniques, and classification models. It analyses the strengths and limitations of Signature-based and Anomaly-based IDS, highlighting the need for improved accuracy and efficiency. Prior research on Principal Component Analysis (PCA) for dimensionality reduction and Random Forest for intrusion classification is examined to justify their integration. The review helps identify research gaps, forming the foundation for the proposed IDS framework.

Pushpa Bharti Singh, Parul Tomar, Madhumitha Kathuria [1] "A Comparative Study of Machine Learning Techniques for Intrusion Detection Systems." This study explores various machine learning techniques used in Intrusion Detection Systems (IDS) and compares their effectiveness in detecting and classifying different types of attacks. The paper highlights the advantages of using a combination of Principal Component Analysis (PCA) and Random Forest, emphasizing how PCA enhances feature selection by reducing irrelevant data, while Random Forest improves classification accuracy through its ensemble learning approach. The research underscores the significance of robust network-level security by integrating these two techniques, ultimately providing an efficient and scalable IDS solution.

Mareddy Sai Krishna Reddy, Kunala Devarsh, Sai Manas Rao Pulakonti [2] "Analysis of Big Data Intrusion using Random Forest." This paper investigates the effectiveness of Random Forest in analyzing large-scale intrusion detection data. The study highlights how deep learning networks, when integrated with IDS, can provide advanced, adaptive, and accurate detection mechanisms for complex intrusion patterns. By leveraging extensive datasets, the research emphasizes how intelligent IDS frameworks can dynamically detect evolving cyber threats and adapt to new attack vectors. The authors suggest that deep learning algorithms offer enhanced cybersecurity by reducing false positives, improving detection speed, and providing better generalization for real-world applications.

G Yedukondalu, G Hima Bindu, J Pavan, G Venkatesh, A Sai Teja [3] “Intrusion Detection System Framework using Machine Learning.” This paper presents a comparative analysis of different machine learning models in intrusion detection, with a specific focus on Artificial Neural Networks (ANN) and Support Vector Machines (SVM). The findings indicate that ANN-based models outperform SVM by achieving an accuracy of 97%. However, the study lacks an in-depth discussion on false positives and false negatives, which are crucial for assessing IDS reliability. The paper underscores the importance of improving IDS frameworks by incorporating feature selection and optimizing hyperparameters to minimize errors while maximizing detection rates.

Satish Kumar, Sunanda Gupta, Sakshi Arora [4] “Research Trends in Network-based Intrusion Detection Systems.” This study provides a comprehensive review of recent research trends in Network-based Intrusion Detection Systems (NIDS). The authors analyze various machine learning-based IDS techniques and discuss widely used datasets, such as KDD Cup 99 and NSL-KDD, which serve as benchmarks for evaluating IDS performance. The research offers valuable insights into the evolution of IDS methodologies, identifying promising advancements and future directions in intrusion detection. The study is beneficial for researchers and beginners in the field, as it provides a structured overview of the latest developments and challenges in NIDS.

Sumit Varshney, Shikha, Shefali Singhi, Bharti Sharma [5] “Intelligent Intrusion Detection System Using Deep Learning Methods.” This paper discusses the application of deep learning models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Autoencoders in IDS. The research highlights how deep learning techniques can enhance intrusion detection by identifying sophisticated attack patterns that traditional machine learning models might miss. The study suggests that IDS frameworks integrating deep learning methods provide better anomaly detection, faster response times, and improved accuracy in identifying potential threats. The authors advocate for a hybrid approach that combines traditional machine learning with deep learning for enhanced cybersecurity threat management.

Usman Shuaibu Musa, Megha Chhabra, Aniso Ali, Mandeep Kaur [6] “Intrusion Detection System using Machine Learning Techniques.” This research compares the performance of various IDS models using single, hybrid, and ensemble machine learning classifiers. The study identifies trends in detection accuracy, highlighting how ensemble-based models tend to outperform single classifiers due to their ability to handle high-dimensional data and improve detection rates.

Jafar Abo Nada and Mohammad Rasmi Al-Mosa [7] “A Proposed Wireless Intrusion Detection Prevention and Attack System.” This paper addresses the limitations of traditional network security mechanisms in wireless environments and proposes a wireless intrusion detection prevention and attack system. The study emphasizes the importance of real-time monitoring, data analysis, and automated defense mechanisms to identify and mitigate wireless threats. It specifically targets denial-of-service attacks, rogue access points, and unauthorized network intrusions. The proposed system incorporates anomaly detection techniques and signature-based detection to enhance network security. The authors highlight the significance of integrating intrusion prevention measures to complement detection mechanisms for a more robust security solution.

Chuanlong Yin, Yuefei Zhu, Jinlong Fei, Xinzheng He [8] “A Deep Learning Approach for Intrusion Detection using Recurrent Neural Networks.” This research presents a deep learning-based IDS utilizing Recurrent Neural Networks (RNN) for both binary and multi-class intrusion detection. The study demonstrates that RNN-based IDS significantly outperform traditional machine learning models in terms of accuracy and efficiency. By tuning hyperparameters such as the number of neurons and learning rates, the authors achieve substantial improvements in intrusion detection. The paper underscores the potential of deep learning in cybersecurity, advocating for the integration of RNNs in IDS frameworks to enhance detection capabilities and minimize false alarms.

Zain Ashi, Laila M. Aburashed, Mohammed Al-Qudah, Abdallah Qusef [9] “Network Intrusion Detection Systems Using Supervised Machine Learning Classification and Dimensionality Reduction Techniques.” This review explores how supervised machine learning algorithms combined with dimensionality reduction techniques can enhance IDS performance. The study examines classifiers such as Naïve Bayes, K-Nearest Neighbors, and Decision Trees, evaluating their effectiveness in different network scenarios. The research also highlights advancements in feature selection methods, including PCA and Linear Discriminant Analysis (LDA), to optimize IDS accuracy and computational efficiency. The findings provide valuable insights into improving detection frameworks by leveraging supervised learning and dimensionality reduction techniques.

Faisal Nabi and Xujuan Zhou [10] “Enhancing Intrusion Detection Systems through Dimensionality Reduction: Machine Learning Techniques for Cyber Security.” This study compares Principal Component Analysis (PCA) and Random Projection (RP) for improving IDS efficiency. The research suggests combining dimensionality reduction with ensemble classifiers for optimal intrusion detection.

CHAPTER 3

PROJECT DESCRIPTION

3.1 EXISTING SYSTEM

The existing Intrusion Detection Systems (IDS) primarily rely on traditional machine learning models and rule-based techniques for detecting cyber threats. These methods often struggle with high-dimensional data, leading to increased computational complexity and slower detection times. Additionally, they suffer from higher false positive and false negative rates, reducing the overall reliability of intrusion detection. Many conventional IDS models fail to adapt to evolving attack patterns, making them less effective against sophisticated cyber threats. Due to these limitations, there is a need for an advanced IDS framework that improves accuracy, efficiency, and adaptability.

3.2 PROPOSED SYSTEM

The novel idea of this project lies in integrating Principal Component Analysis (PCA) with the Random Forest algorithm to enhance the accuracy, efficiency, and scalability of Intrusion Detection Systems (IDS). By incorporating PCA, the dataset's dimensionality is significantly reduced while preserving essential features, leading to faster data processing, lower computational overhead, and improved interpretability. Once optimized, the Random Forest algorithm classifies network traffic and detects anomalies using multiple decision trees, improving classification accuracy and robustness. This ensemble learning technique ensures resistance to overfitting, making the IDS more reliable. The hybrid approach of PCA and Random Forest enhances detection rates while minimizing false positives, ensuring that normal network activities are not wrongly classified as attacks.

3.2.1 ADVANTAGES

- Enhances intrusion detection accuracy by removing irrelevant features.
- Reduces computational load with PCA, ensuring faster processing and efficiency.
- Increases model robustness, making it resistant to overfitting.
- Lowers false positives, improving precision in anomaly detection.
- Scales efficiently for large datasets and real-time cybersecurity applications.
- Extends usability to malware detection and fraud prevention.

3.3 FEASIBILITY STUDY

A feasibility study is conducted to assess the viability of the project and analyze its strengths and weaknesses. In this context, the feasibility study is conducted across three dimensions:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.3.1 ECONOMIC FEASIBILITY

The proposed IDS framework integrating PCA and Random Forest is economically viable as it optimizes resource utilization and minimizes computational costs. By reducing the dimensionality of data, PCA decreases storage and processing overhead, making it cost-effective for deployment in real-world cybersecurity environments. Additionally, the use of Random Forest, an efficient ensemble learning technique, ensures high accuracy without requiring expensive computational power. The model's scalability allows organizations to implement it within existing infrastructures, reducing the need for costly hardware upgrades or additional software investments.

3.3.2 TECHNICAL FEASIBILITY

The integration of PCA and Random Forest is technically feasible due to their compatibility with modern machine learning frameworks and cybersecurity infrastructures. PCA efficiently reduces high-dimensional datasets, enabling faster processing and better model performance. Random Forest, known for its robustness and adaptability, effectively classifies network traffic and identifies anomalies.

3.3.3 SOCIAL FEASIBILITY

By reducing false positives and improving anomaly detection, the system ensures a safer digital environment for organizations and individuals. Furthermore, the efficient threat detection system minimizes cyber threats, reducing financial and personal losses associated with data breaches.

3.4 SYSTEM SPECIFICATION

An effective system is crucial for any computational task. It's important to have the correct hardware and software components to ensure everything runs smoothly. From strong processors to essential software packages, each part helps create an efficient environment for data analysis and machine learning tasks

3.4.1 HARDWARE SPECIFICATION

- Processor: Intel Core or higher with i5, i7 or i9
- Ethernet connection (LAN) OR a wireless adapter (Wi-Fi)
- Hard Drive: Minimum 100 GB; Recommended 250 GB or more
- Memory (RAM): Minimum 8 GB; Recommended 16 GB or above

3.4.2 SOFTWARE SPECIFICATION

- Visual Studio Code
- Jupyter Notebook
- OpenCV-Python
- Notepad

CHAPTER 4

PROPOSED WORK OF

PCA AND RANDOM FOREST

4.1 GENERAL ARCHITECTURE

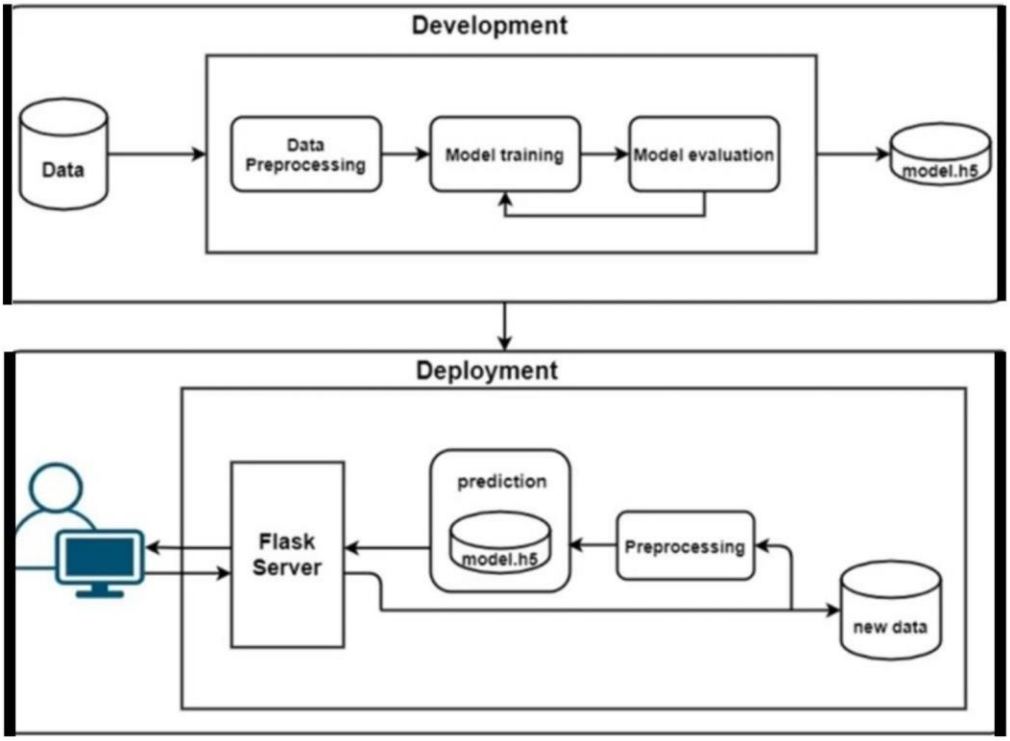


Figure 4.1: **Architecture Diagram of Proposed Work.**

Figure 4.1 illustrates a machine learning model pipeline for development and deployment. Raw data undergoes preprocessing, including cleaning and normalization, before training a machine learning algorithm. The trained model is evaluated using metrics like accuracy and F1-score, then saved as 'model.h5' upon optimization. In deployment, a Flask server acts as an API to handle user inputs, preprocess them, and feed them into the saved model for prediction. The prediction is returned to the user interface for real-time inference. This structured approach ensures seamless integration, scalability, and efficiency. The model remains reusable and can be updated with new data through retraining and redeployment.

4.2 DESIGN PHASE

During the design phase, diverse diagrams and models are crafted to depict various elements of the system, such as its components, interactions, and data flow. These diagrams, including UML, sequence, use case, and data flow diagrams, aid in conveying the system's design and functionality to stakeholders and development teams. In essence, the design phase is pivotal for ensuring that the software solution achieves its objectives in a proficient and effective manner.

4.2.1 DATA FLOW DIAGRAM

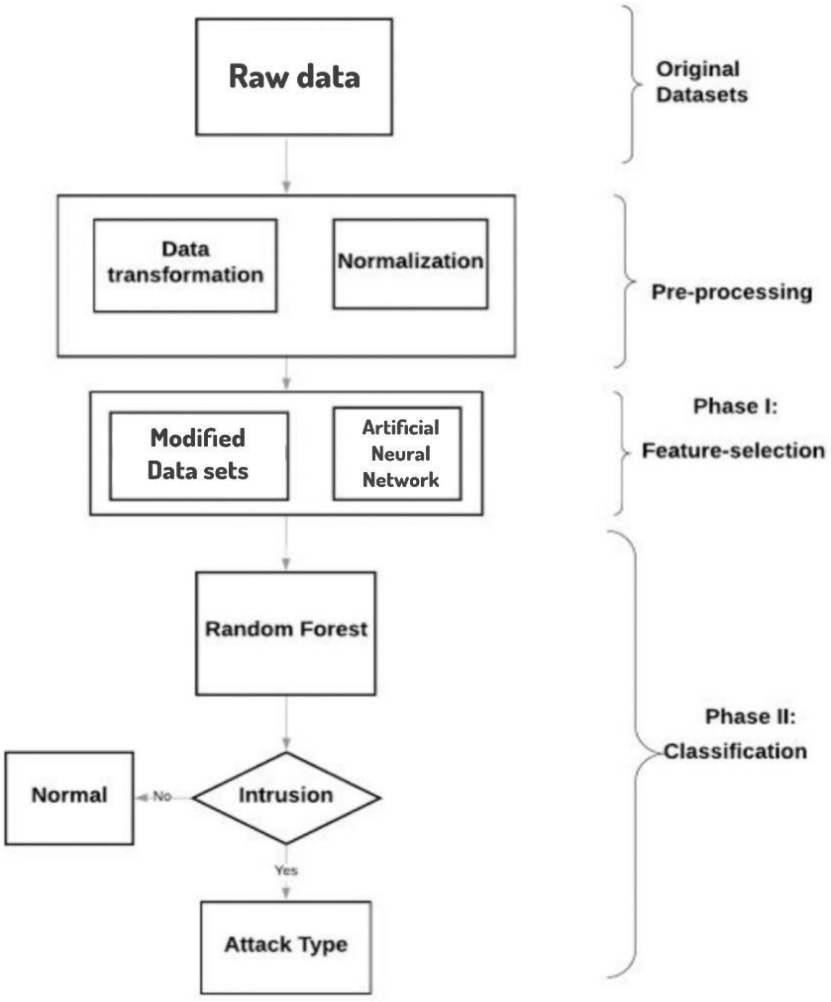


Figure 4.2: **Data Flow Diagram of Proposed Work.**

The Figure 4.2 shows that Intrusion Detection System (IDS) integrates Artificial Neural Networks (ANN) for feature selection and Random Forest for classification. Pre-processed network traffic data undergoes ANN-based feature extraction to reduce redundancy and improve efficiency. The refined data is then classified using random forest to detect intrusions and categorize attack types for targeted security responses. This hybrid approach enhances accuracy, minimizes false positives, and optimizes computational performance.

4.2.2 UML DIAGRAM

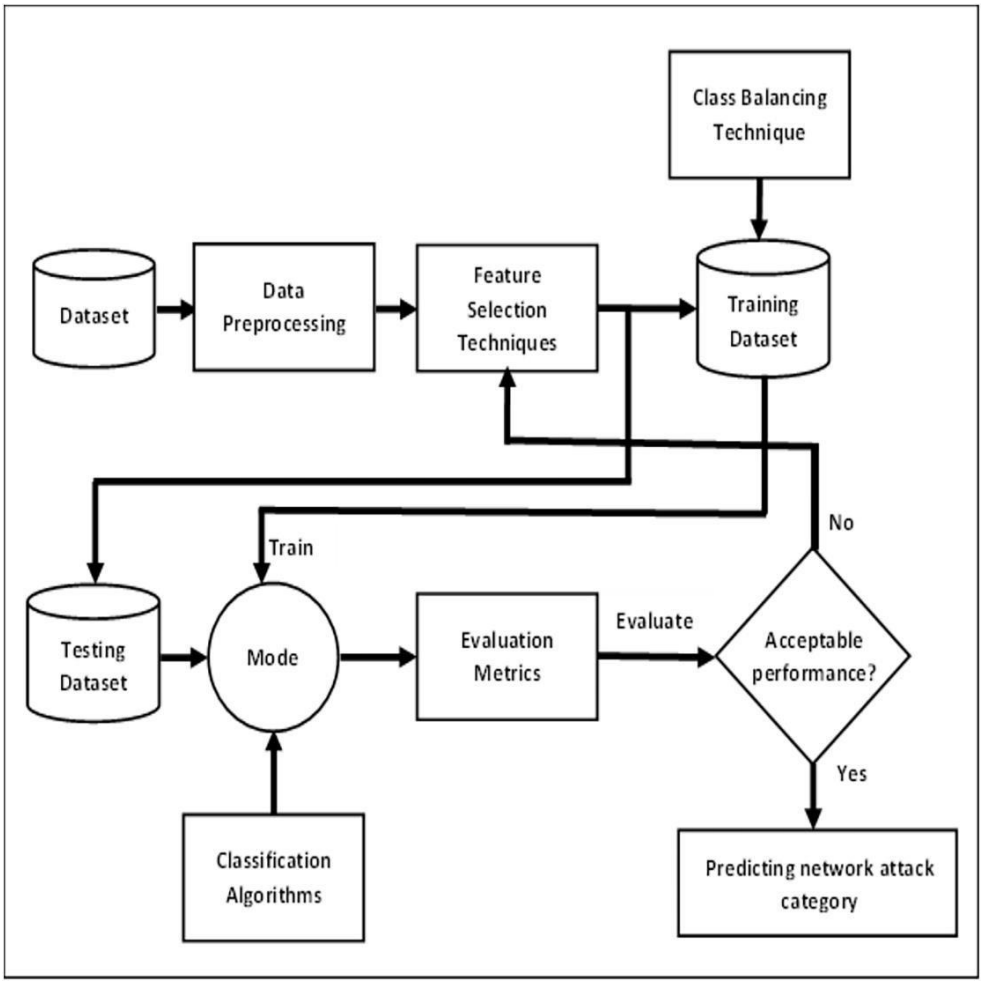


Figure 4.3: **UML Diagram of Proposed Work.**

The Figure 4.3 is a UML diagram representing an Intrusion Detection System (IDS) framework using Principal Component Analysis (PCA) and Random Forest for network attack classification. The process begins with a dataset, which undergoes data preprocessing to clean and prepare the data. PCA is applied to reduce dimensionality and select the most significant features. The training dataset is used to train a model with classification algorithms, such as Random Forest, while a testing dataset evaluates performance. The trained model is assessed using evaluation metrics, and if the performance is not acceptable, the feature selection and training process are repeated. Once an optimal model is achieved, the IDS is capable of accurately predicting network attack categories in real time. This approach enhances detection accuracy and computational efficiency while reducing false positives.

4.2.3 USE CASE DIAGRAM

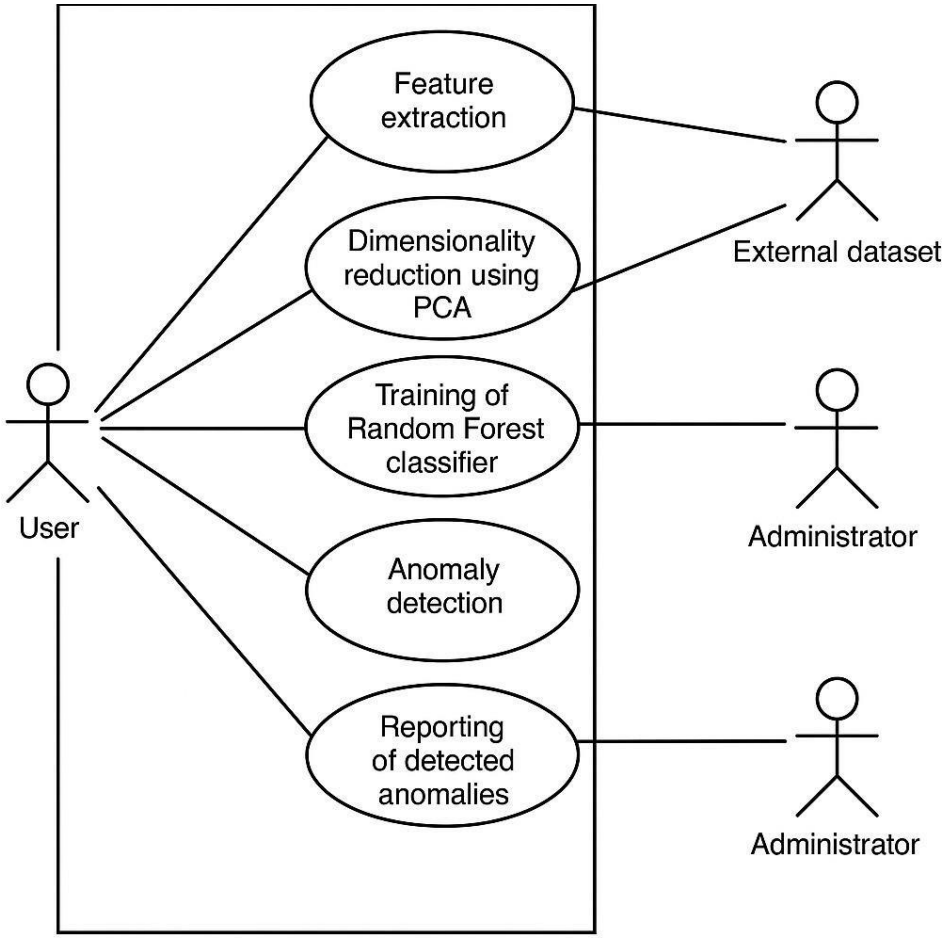


Figure 4.4: **Use Case Diagram of Proposed Work.**

The diagram in Figure 4.4 illustrates the Use Case Diagram of Intrusion Detection System (IDS) framework utilizing Principal Component Analysis (PCA) and Random Forest for anomaly detection. The user initiates the process by performing Feature Extraction, which gathers relevant characteristics from an external dataset. The system then applies dimensionality reduction using PCA to eliminate redundant features and enhance computational efficiency. Next, the Random Forest classifier is trained, to accurately distinguish between normal and malicious activities. The trained model is used for anomaly detection, identifying potential cyber threats in real-time. Finally, detected anomalies are reported to the Administrator, ensuring timely response and mitigation of cyber threats.

4.2.4 SEQUENCE DIAGRAM

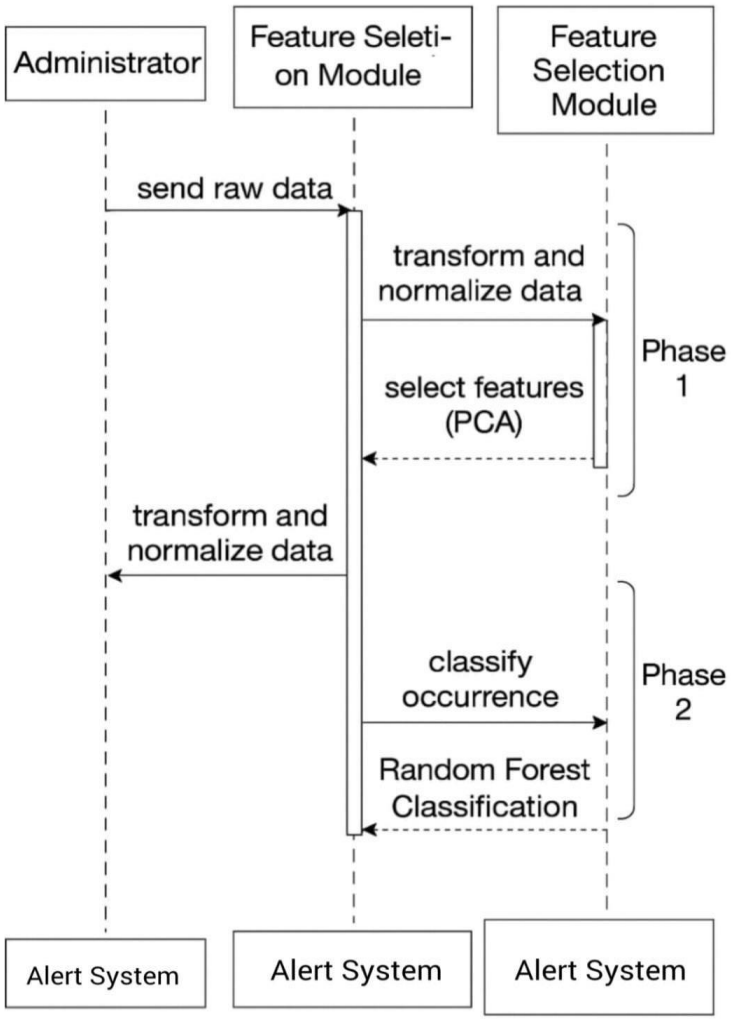


Figure 4.5: **Sequence Diagram of Proposed Work.**

This sequence diagram represents the workflow of an Intrusion Detection System (IDS) using PCA and Random Forest. The Administrator initiates the process by sending raw data to the Feature Selection Module, which performs data transformation and normalization. The Feature Selection Module applies Principal Component Analysis (PCA) to extract the most relevant features (Phase 1). The transformed data is then passed to the Random Forest classification module, where occurrences are classified as normal or anomalous (Phase 2). If an anomaly is detected, the system generates an alert through the Alert System. This systematic approach enhances the efficiency of intrusion detection by improving feature selection and classification accuracy.

4.3 MODULE DESCRIPTION

The system consists of three core modules: Data Preprocessing, Dimensionality Reduction using PCA, and Classification using Random Forest. These modules work together to filter data, reduce complexity, and accurately detect intrusions in network traffic.

4.3.1 MODULE1 : DATA PREPROCESSING

The Data Preprocessing module plays a vital role in ensuring the accuracy and reliability of the Intrusion Detection System (IDS). Raw network traffic data contains inconsistencies such as missing values, duplicate entries, and irrelevant features that can negatively impact model performance. This module addresses these issues by cleaning and structuring the data to improve its quality. Thus, this module lays a strong foundation for effective dimensionality reduction and accurate intrusion detection

4.3.2 MODULE 2 : DIMENSIONALITY REDUCTION USING PCA

The Dimensionality Reduction module utilizes Principal Component Analysis (PCA) to address the complexity and redundancy often present in high-dimensional network security datasets. PCA works by identifying and retaining only the most significant features that contribute to the variance in the data, effectively filtering out irrelevant attributes. This reduction improves computational efficiency by lowering the training time required for machine learning models. Additionally, it enhances the interpretability of the system by focusing on the most meaningful data patterns, ultimately contributing to more accurate and faster intrusion detection.

4.3.3 MODULE 3: CLASSIFICATION USING RANDOM FOREST

The Classification module employs the Random Forest algorithm to categorize network traffic into normal activity or specific types of attacks. After the dataset is simplified through PCA, Random Forest processes the refined features by constructing multiple decision trees during training. Each tree independently analyzes patterns in the data, and their collective outputs are aggregated to make final predictions. This ensemble approach enhances classification accuracy, reduces the risk of overfitting, and ensures robust performance even with complex and varied intrusion patterns, making it highly effective for reliable intrusion detection.

4.3.4 STEP 2: PROCESSING OF DATA

- Data preprocessing is a crucial step in any machine learning or data analysis pipeline, as it transforms raw data into a clean and usable format for modeling. This process begins by importing essential libraries such as `pandas` and `numpy`, which aid in data manipulation and numerical operations.
- The dataset is then loaded, often using `pd.read_csv()`, and examined for inconsistencies or missing values. Missing values are either removed using methods like `dropna()` or filled using techniques like forward fill (`fillna(method='ffill')`).
- Next, categorical variables are those containing text or labels, which are converted into numerical representations using label encoding or one-hot encoding, allowing models to interpret them effectively.
- Feature scaling is then performed using tools like `StandardScaler` from `sklearn`, which standardizes numerical columns so that each feature contributes equally to the model's learning.
- Finally, the cleaned and transformed dataset is split into training and testing sets using `train_test_split`, ensuring that the model can be trained and evaluated on separate portions of the data.
- Altogether, these preprocessing steps lay the foundation for robust and accurate machine learning models by addressing noise, inconsistency, and scale disparities in the data.

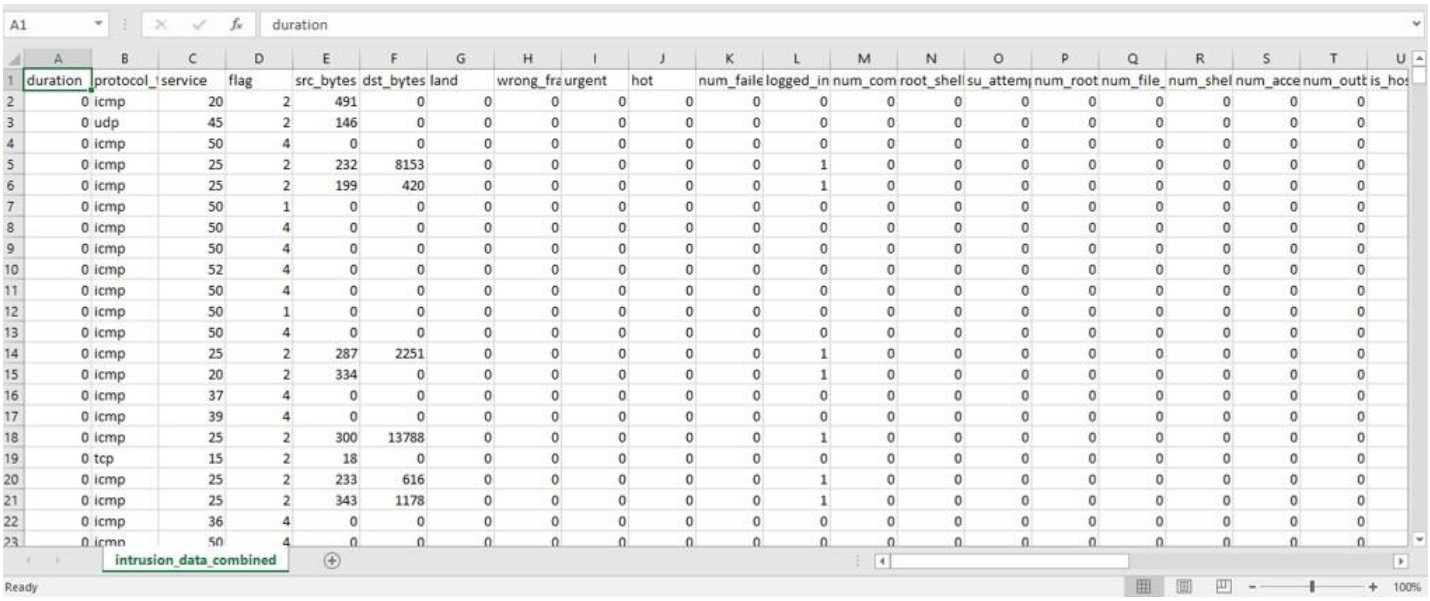
```python
# Importing necessary libraries
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv('/content/loan_data.csv')

# Drop unnecessary columns
df = df.drop(['Loan_ID'], axis=1)

# Fill missing values
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df['Married'].fillna(df['Married'].mode()[0], inplace=True)
df['Dependents'].fillna(df['Dependents'].mode()[0], inplace=True)
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0], inplace=True)
df['LoanAmount'].fillna(df['LoanAmount'].mean(), inplace=True)
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0], inplace=True)
df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)

# Convert categorical variables to numerical using Label Encoding
cols = ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area', 'Loan_Status']
le = LabelEncoder()
for col in cols:
    df[col] = le.fit_transform(df[col])

# Convert 'Dependents' column to integer after replacing '3+' with 3
df['Dependents'] = df['Dependents'].replace('3+', 3).astype(int)

# Split data into features and target
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Figure 4.6: **Pre-processing Of Data using Python.**

4.3.5 STEP 3: SPLIT THE DATA

- The dataset is divided into input features (x) and the output label (y).
- The data is split into 80% training and 20% testing using train_test_split() to evaluate model performance on unseen data.
- A random_state is set to ensure consistent splitting every time the code runs, allowing reproducibility of results.

4.3.6 DATASETS SAMPLE



duration	protocol_t	service	flag	src_bytes	dst_bytes	land	wrong_fra	urgent	hot	num_faile	logged_in	num_com	root_shell	su_attem	num_root	num_file_	num_shel	num_acce	num_outb	is_hos
0	icmp	20	2	491	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	udp	45	2	146	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	25	2	232	8153	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	25	2	199	420	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	50	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	52	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	25	2	287	2251	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	20	2	334	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	37	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	39	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	25	2	300	13788	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	tcp	15	2	18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	25	2	233	616	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	25	2	343	1178	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0	icmp	36	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	icmp	50	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Figure 4.7: **Input Datasets for Intrusion Detection**

```
data.head()
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot
0	0	icmp	20	2	491	0	0	0	0	0
1	0	udp	45	2	146	0	0	0	0	0
2	0	icmp	50	4	0	0	0	0	0	0
3	0	icmp	25	2	232	8153	0	0	0	0
4	0	icmp	25	2	199	420	0	0	0	0

5 rows × 42 columns

Figure 4.8: **Overview of Network Data**

4.3.7 STEP 4: BUILDING THE MODEL

Our model of choice is a PCA-enhanced Random Forest classifier, implemented using the Scikit-learn library in Python, developed and tested using Visual Studio Code and Google Colab.

INPUT LAYER:

The input layer of the model receives a vector of selected network traffic features, preprocessed and reduced using PCA to form a fixed-length input for classification.

ENCODER SECTION:

The encoder section applies PCA to reduce the input feature dimensions while preserving important patterns. This step compresses the data, making it cleaner and faster for the Random Forest to classify.

DECODER SECTION:

In our model, the decoder role is handled by the Random Forest classifier. It takes the compressed features from PCA and maps them to specific output classes. These outputs indicate whether the network activity is normal or a type of malicious attack.

MODEL CREATION:

Finally, the PCA output and Random Forest classifier are combined to form a complete intrusion detection model ready for training and evaluation.

dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate
25	0.17	0.03
1	0.00	0.60
26	0.10	0.05
255	1.00	0.00
255	1.00	0.00
...	...	...
255	1.00	0.00
253	1.00	0.00
254	1.00	0.01
255	1.00	0.00
14	0.05	0.84

Figure 4.9**: Training and Evaluation of the Model**

4.3.8 STEP 5: COMPILING AND TRAINING THE MODEL

Model performance is evaluated using key metrics such as accuracy, precision, recall, and F1-score. Multiple training runs were conducted to fine-tune the number of estimators and tree depth for best results.

The accuracy of the model is evaluated using the accuracy_score function, and a detailed performance summary including precision, recall, and F1-score is generated through the classification_report as mentioned in Figure 4.10.

```
# Step 5: Apply PCA for feature reduction
pca = PCA(n_components=10)  # Adjust the number of components based on your dataset
```

```
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

Figure 4.10: **Code to Compile Model and Fit Model to Dataset**

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 INPUT AND OUTPUT

5.1.1 DATA REPRESENTATION

The input values shown are fed into the prediction form of the intrusion detection model, where each row represents a network activity instance. Upon submission, the model processes these inputs and classifies them as either normal traffic or a malicious intrusion, effectively validating the system's detection capabilities in real-world-like scenarios.

Figure 5.1 below shows the model input values.

duration	protocol_type	service	flag	src_bytes	dst_bytes
0	icmp	20	2	491	0
0	udp	45	2	146	0
0	icmp	50	4	0	0
0	icmp	25	2	232	8153
0	icmp	25	2	199	420
0	icmp	50	1	0	0
0	icmp	50	4	0	0
0	icmp	50	4	0	0
0	icmp	52	4	0	0
0	icmp	50	4	0	0
0	icmp	50	1	0	0
0	icmp	50	4	0	0
0	icmp	25	2	287	2251
0	icmp	20	2	334	0
0	icmp	37	4	0	0
0	icmp	39	4	0	0
0	icmp	25	2	300	13788
0	tcp	15	2	18	0
0	icmp	25	2	233	616
0	icmp	25	2	343	1178
0	icmp	36	4	0	0
0	icmp	50	4	0	0
0	icmp	25	2	253	11905
5607	udp	45	2	147	105
0	icmp	36	4	0	0
507	icmp	61	2	437	14421
0	icmp	50	4	0	0

Figure 5.1: **Model Input Values for Testing the Model.**

5.1.2 PREPROCESSING AND MODEL EVALUATION

Figure 5.2 below shows the predicted output which categorizes as a malicious or non-malicious attack.

```
Accuracy: 0.9730465159036588
Confusion Matrix:
[[ 9756    78    11     0     0]
 [   74 14221    42    12     8]
 [  421    16  2114     2     0]
 [    0    46     1   365     1]
 [    0     9    11     1     6]]
Classification Report:
              precision    recall  f1-score   support

         dos       0.95      0.99      0.97      9845
      normal       0.99      0.99      0.99     14357
       probe       0.97      0.83      0.89      2553
         r2l       0.96      0.88      0.92       413
         u2r       0.40      0.22      0.29        27

    accuracy                           0.97     27195
   macro avg       0.85      0.78      0.81     27195
weighted avg       0.97      0.97      0.97     27195
```

Figure 5.2**: Evaluation of the Model for Proposed System.**

5.2 TESTING

To test the model, a sample dataset containing various network activity parameters is inputted through the prediction form. These inputs are preprocessed and passed through the PCA transformation before being fed into the trained Random Forest classifier. The model then outputs a prediction indicating whether each instance is normal or represents a potential intrusion. This testing approach ensures the model's real-time applicability and verifies its accuracy, precision, and robustness against diverse network behaviors.

5.2.1 TYPES OF TESTING

5.2.2 UNIT TESTING

Unit Testing is performed to individually verify the correctness and performance of each functional component in the intrusion detection system. **Figure 5.3** below contains the code for model preprocessing, implementation and evaluation.

INPUT:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# scaler = StandardScaler()
# X_train_scaled = scaler.fit_transform(x_train)
# X_test_scaled = scaler.transform(x_test)

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder())
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Step 5: Apply PCA for feature reduction
pca = PCA(n_components=10)  # Adjust the number of components based on your dataset

# Step 6: Create and train the classifier (Random Forest)
classifier = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('pca', pca),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
])
```

```
X = data[['duration','service','flag','src_bytes','dst_bytes','land','wrong_fragment','urgent','hot','num_failed_logins']]
Y = data['label']
```

```
X.shape
Y.shape
```

```
(135973,)
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=42)
```

```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Figure 5.3: **Model Preprocessing and Model Evaluation of Proposed System.**

TEST RESULT

- The Intrusion Detection System achieved an overall accuracy of 97.3% in classifying network traffic.
- Major classes like normal and dos showed high precision and recall.
- The average F1-score is 0.81, while the weighted average F1-score is 0.97, indicating strong overall accuracy.

5.2.3 INTEGRATION TESTING

INPUT:

```
# Assuming 'new_data' is a list of 10 values for the features you want to predict
new_data = [0,20,2,334,0,0,0,0,0,0]

# Create a new dataframe for the new data
new_data_df = pd.DataFrame([new_data], columns=['duration', 'service', 'flag', 'src_bytes', 'dst_bytes',
                                                'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins'])

# Use the trained classifier to make predictions on the new data
y_pred_new_data = classifier.predict(new_data_df)

# Check if the predicted label is normal (0) or malware (1)
if y_pred_new_data[0] == "normal":
    print("The data is predicted as normal.")
elif y_pred_new_data[0] =="dos":
    print("The data is predicted as dos.")
elif y_pred_new_data[0]=="probe":
    print("The data is predicted as probe.")
elif y_pred_new_data[0]=="r2l":
    print("The data is predicted as r2l.")
```

Figure 5.4: **Prediction Data Classification of Proposed System.**

TEST RESULT

- Integration testing is performed by inputting a new set of feature values into the trained classifier to verify accurate real-time predictions.
- The output confirms that the system correctly classifies the input as normal or a specific attack type like DoS, Probe, or R2L

5.2.4 FUNCTIONAL TESTING

INPUT:

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=42)
```

```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Figure 5.5: **Training and Testing Split of Data Structures.**

```
data=pd.read_csv('intrusion_data_combined.csv')
data
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes
0	0	icmp	20	2	491	0
1	0	udp	45	2	146	0
2	0	icmp	50	4	0	0
3	0	icmp	25	2	232	8153
4	0	icmp	25	2	199	420
...	...	...	...	...	...	...
135968	0	icmp	25	2	315	2537
135969	0	icmp	25	2	238	6882
135970	0	udp	50	2	54	55
135971	0	icmp	25	2	228	5210
135972	282	icmp	20	2	160	597

135973 rows × 42 columns

Figure 5.6: **Model is Compiled and Trained for Proposed System.**

TEST RESULT

- All network traffic records from the dataset were loaded into the model and used for training and testing purposes.
- Each record was evaluated to check if the classifier correctly identified and labeled it based on learned patterns.

CHAPTER 6
RESULTS AND DISCUSSIONS

6.1 EFFICIENCY OF THE PROPOSED SYSTEM

The proposed Intrusion Detection System (IDS) demonstrates high efficiency by effectively reducing dimensionality using PCA, which lowers computation time and memory usage without losing critical data. The Random Forest algorithm ensures accurate classification by aggregating results from multiple decision trees. This hybrid model enhances detection rates and minimizes false positives, improving reliability. It handles large-scale datasets efficiently and adapts well to varying data distributions. The system shows robustness against overfitting and performs consistently across different attack categories. Its preprocessing module improves input data quality, which contributes to better predictions. Real-time detection is achievable due to the streamlined data flow and model response time. The architecture supports scalability and easy deployment, making it practical for real-world scenarios. Overall, the system's combination of speed, accuracy, and adaptability results in a highly efficient and secure network protection framework.

6.2 COMPARISON OF EXISTING AND PROPOSED SYSTEM

The existing system for intrusion detection often struggles with high-dimensional data, leading to increased processing time and reduced accuracy. It relies on manual feature selection, which can overlook critical attributes and increase false positives. In contrast, the proposed system leverages Principal Component Analysis (PCA) to automatically reduce dimensionality, retaining only the most relevant features. While traditional methods may use a single classifier, the proposed model utilizes Random Forest, an ensemble learning technique, to boost classification accuracy and robustness. Additionally, the proposed system demonstrates improved detection rates across multiple attack types, including rare ones like Remote-to-Local (R2L) attack and User-to-Root (U2R) attack. Deployment and real-time detection are also more efficient in the proposed system due to streamlined preprocessing and prediction pipelines. Overall, the proposed system provides a more scalable, accurate, and reliable solution compared to conventional IDS approaches.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The proposed Intrusion Detection System (IDS) successfully integrates Principal Component Analysis (PCA) with the Random Forest algorithm to create a robust, efficient, and accurate framework for detecting cyber threats. By reducing the dimensionality of high-volume network traffic data, PCA enhances model performance while minimizing computational costs. The use of Random Forest ensures high classification accuracy as the system effectively distinguishes between normal activity and malicious attack types, reducing false positives and improving overall network security. In conclusion, the project demonstrates a scalable and intelligent solution that strengthens the capabilities of modern intrusion detection systems in real-time environments.

7.2 FUTURE ENHANCEMENTS

Future enhancements in Intrusion Detection System can be expanded through its adaptability to larger and more complex network environments. The system's performance can be improved by refining feature selection and optimizing parameter tuning for better detection accuracy. It can also be go through an advanced approach for real-time intrusion monitoring, ensuring faster responses to security breaches. Continuous updating with new intrusion patterns will help maintain its relevance and effectiveness against emerging threats.

7.3 RESULTS:

Classification Report:

	precision	recall	f1-score	support
dos	0.95	0.99	0.97	9845
normal	0.99	0.99	0.99	14357
probe	0.97	0.83	0.89	2553
r2l	0.96	0.88	0.92	413
u2r	0.40	0.22	0.29	27
accuracy			0.97	27195
macro avg	0.85	0.78	0.81	27195
weighted avg	0.97	0.97	0.97	27195

Figure 7.1: **Random Forest Evaluation Model of Proposed System.**

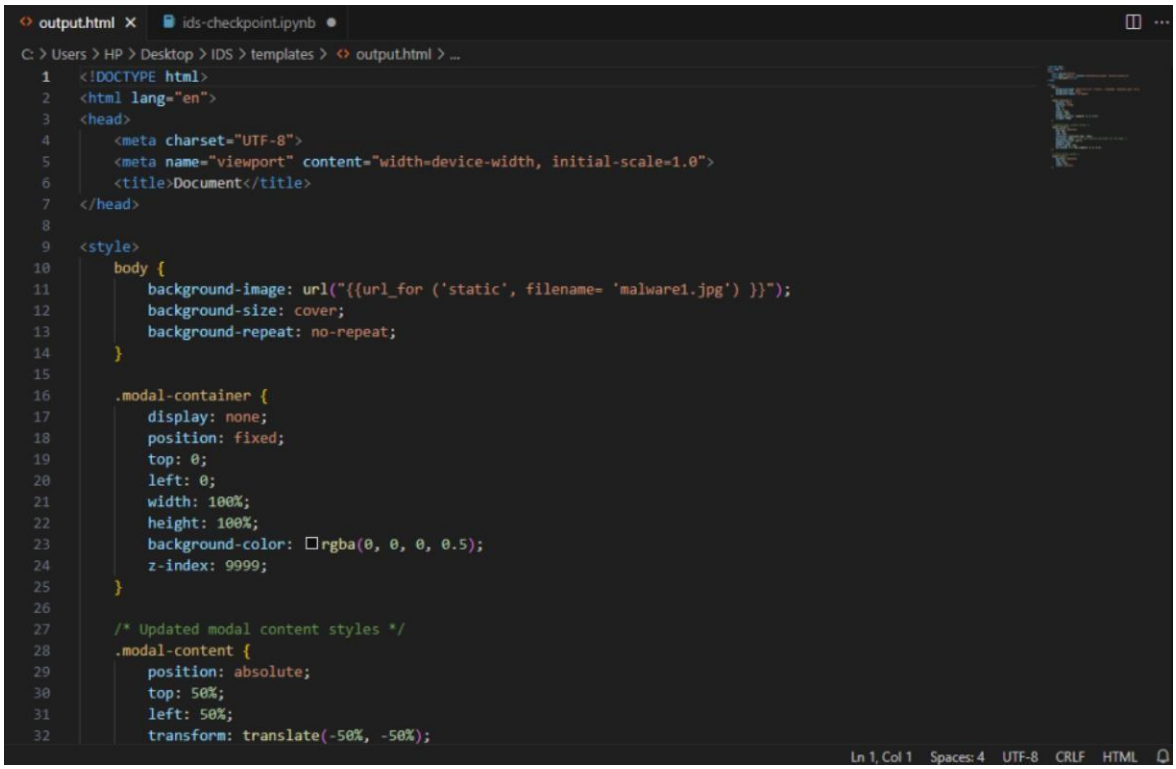


Figure 7.2: **Output Model Prediction of Proposed System.**

CHAPTER 8

SOURCE CODE OF PROPOSED SYSTEM

8.1 SOURCE CODE

```
import numpy as np
import pandas as pd
```

```
data=pd.read_csv('intrusion_data_combined.csv')
data
```

service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate
20	2	491	0	0	0	0	0	...	25	0.17	0.03	0.17
45	2	146	0	0	0	0	0	...	1	0.00	0.60	0.88
50	4	0	0	0	0	0	0	...	26	0.10	0.05	0.00
25	2	232	8153	0	0	0	0	...	255	1.00	0.00	0.03
25	2	199	420	0	0	0	0	...	255	1.00	0.00	0.00
...	...	...	...	...	...	...	...	...	...	...	...	...
25	2	315	2537	0	0	0	0	...	255	1.00	0.00	0.00
25	2	238	6882	0	0	0	0	...	253	1.00	0.00	0.14
50	2	54	55	0	0	0	0	...	254	1.00	0.01	0.81
25	2	228	5210	0	0	0	0	...	255	1.00	0.00	0.01
20	2	160	597	0	0	0	2	...	14	0.05	0.84	0.00

```
data.head()
```

service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate	dst_host_sr
20	2	491	0	0	0	0	0	...	25	0.17	0.03	0.17	
45	2	146	0	0	0	0	0	...	1	0.00	0.60	0.88	
50	4	0	0	0	0	0	0	...	26	0.10	0.05	0.00	
25	2	232	8153	0	0	0	0	...	255	1.00	0.00	0.03	
25	2	199	420	0	0	0	0	...	255	1.00	0.00	0.00	

```
data.shape
```

```
(135973, 42)
```

```
data.isnull().values.any()
```

```
False
```

```
data.isnull().sum()
```

```
X = data[['duration','service','flag','src_bytes','dst_bytes','land','wrong_fragment','urgent','hot','num_failed_logins']]
Y = data['label']
```

```
X.shape
Y.shape
```

```
(135973,)
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=42)
```

```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(108778, 10)
(27195, 10)
(108778,)
(27195,)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# scaler = StandardScaler()
# X_train_scaled = scaler.fit_transform(x_train)
# X_test_scaled = scaler.transform(x_test)

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder())
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])
```

```
y_pred = classifier.predict(x_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(report)
```

```
# Assuming 'new_data' is a list of 10 values for the features you want to predict
new_data = [0,20,2,334,0,0,0,0,0,0]

# Create a new dataframe for the new data
new_data_df = pd.DataFrame([new_data], columns=['duration', 'service', 'flag', 'src_bytes', 'dst_bytes',
                                                'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins'])

    # Use the trained classifier to make predictions on the new data
    y_pred_new_data = classifier.predict(new_data_df)

    # Check if the predicted label is normal (0) or malware (1)
    if y_pred_new_data[0] == "normal":
        print("The data is predicted as normal.")
    elif y_pred_new_data[0] =="dos":
        print("The data is predicted as dos.")
    elif y_pred_new_data[0]=="probe":
        print("The data is predicted as probe.")
    elif y_pred_new_data[0]=="r2l":
        print("The data is predicted as r2l.")
```

```
The data is predicted as r2l.
```

```
import pickle
# Specify the file path where you want to save the pickle file
file_path = 'classifier.pkl'

# Open the file in binary write mode
with open(file_path, 'wb') as file:

    pickle.dump(classifier, file)
```

REFERENCES

[1] Pushpa Bharti Singh, Parul Tomar, Madhumitha Kathuria "A Comparative Study of Machine Learning Techniques for Intrusion Detection Systems", COM-IT-CON, May 2022.

[2] Mareddy Sai Krishna Reddy, Kunala Devarsh, Sai Manas Rao Pulakonti, "Hayato Analysis of Big Data Intrusion using Random Forest", Publisher: IRJMETS, November 2022.

[3] G Yedukondalu, G Hima Bindu, J Pavan, G Venkatesh, A Sai Teja, "Intrusion Detection System Framework using Machine Learning", Publisher: ICIRCA September 2021.

[4] Satish Kumar, Sunanda Gupta, Sakshi Arora, "Research Trends in Network-based Intrusion Detection", Publisher: IEEE, November 2021.

[5] Sumit Varshney, Shikha, Shefali Singhi, Bharti Sharma, "Intelligent Intrusion Detection System Using Deep Learning Methods", Publisher: IEEE, June 2021.

[6] Usman Shuaibu Musa, Megha Chhabra, Aniso Ali, Mandeep Kaur, "Intrusion Detection System using Machine Learning", Publisher: IEEE, October 2020.

[7] Jafar Abo Nada and Mohammad Rasmi Al-Mosa, "A Proposed Wireless Intrusion Detection Prevention and Attack", Publisher: IEEE, March 2019.

[8] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, Xinzheng He, "A Deep Learning Approach for Intrusion Detection using Recurrent Neural", Publisher: IEEE, October 2017.

[9] Zain Ashi, Laila M Aburashed, Mohammed Al-Qudah, Abdallah Qusef, "Network Intrusion Detection Systems Using Supervised Machine Learning Classification and Dimensionality Reduction", Publisher: JJCIT, December 2021.

[10] Faisal Nabi and Xujuan Zhou, “Enhancing Intrusion Detection Systems through Dimensionality Reduction: Machine Learning Techniques for Cyber Security”, Publisher: KeAi, January 2024.

[11] Mohammed Tarek Abdelaziz, Abdelrahman Radwan, Hesham Mamdouh, Adel Saeed Saad, Abdulrahman Salem Abuzaid, Ahmed Ayman, Salma Zakzouk, Kareem Moussa and M Saeed Darweesh, “Enhancing Network Threat Detection with Random Forest-Based NIDS and Permutation Feature Importance”, Publisher: Journal of Network and Systems Management, Volume 33, Article 2, October 2024.

[12] Sureswar Reddy Dronadula, Sagar S, Dhruv Gandhi, Siri R Kulakarni and SP Raja, “An Innovative Approach and Evaluation of Contemporary Intrusion Detection Systems”, Publisher: Journal of Cyber Security Technology, November 2024.

[13] SK Shameena Begum, Pulikonda Venkata Yoga Abhishek, Kadavakollu Lakshmi Chaitanya and Mandalapu Gopichand, “Intrusion Detection System Using ANN”, Publisher: International Research Journal on Advanced Engineering Hub (IRJAEH), December 2024.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University u / s 3 of UGC Act, 1956)

Office of Controller of Examinations

REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION / PROJECT REPORT FOR UG / PG PROGRAMMES

(To be attached in the dissertation / project report)

1	Name of the Candidate	ROSHAN KUMAR S MAGESHWARAN SD KRITHIK S R
2	Address of Candidate	Bharathi Salai, Chennai - 600089 **Mobile Number:** +91 82486 73686 +91 90032 21178 +91 98403 26345
3	Registration Number	RA2111030020018, RA2111030020028, RA2111030020047
4	Date of Birth	02/01/2004, 04/07/2003, 16/02/2004
5	Department	Computer Science and Engineering with specialization in Cyber Security
6	Faculty	Engineering and Technology
7	Title of the Dissertation / Project	A Novel IDS Framework Combining PCA and Random Forest
8	Whether the above project / dissertation is done by	Individual or Group: **Group** a) If the project / dissertation is done in group, then how many students together completed the project 03 b) Name and Registration Number of Candidates: ROSHAN KUMAR S [RA2111030018], MAGESHWARAN SD [RA2111030020028], KRITHIK S R [RA2111030020047]

9	Name and address of the Supervisor / Guide	ADDRESS OF GUIDE **Mail ID:** drsurendaru@gmail.com **Mobile Number:** +91 97891 83816

10	Name and address of the Co- Supervisor /Guide	NA **Mail ID:** NA **Mobile Number:** NA		
11	Software Used	Turnitin		
12	Date of Verification			
13	**Plagiarism Details: (to attach the final report from the software)**			
S . NO	**Title of the Report**	**Percentage of similarity index (including self-citation)**	**Percentage of similarity index (Excluding self-citation)**	**% of plagiarism after excluding Quotes, Bibliography, etc.,**
1	A Novel IDS Framework Combining PCA and Random Forest	NA	NA	10%
Appendices		NA	NA	NA
We declare that the above information has been verified and found true to the best of our knowledge.				

Signature of the Candidate	**Name and Signature of the Staff** **(Who uses the plagiarism check software)**
Name and Signature of the Supervisor / Guide	**Name and Signature of the Co-Supervisor / Co-Guide**
Dr. J. Shiny Duela, M.E., Ph.D., Associate Professor and Head of Department, Computer Science and Engineering, SRM Institute of Science and Technology, Ramapuram, Chennai.	

Gmail Krithik Ravichandran <krithuravichandran@gmail.com>

Thanks for your contribution in IJSRET Journal.

1 message

support@ijsret.com <support@ijsret.com> Thu, Apr 10, 2025 at 5:15 PM
Reply-To: support@ijsret.com
To: krithuravichandran@gmail.com

Dear Krithik S R,

Thanks for your contribution in IJSRET Journal.

Your submitted manuscript having title 'A Novel IDS Framework Combining PCA and Random Forest' received successfully. As per paper content editor forward to concern reviewer. You will be informed about paper acceptance status very soon. For more help contact at +91 75666 32455 (Whatsapp).

Regards

Team IJSRET

www.ijsret.com

+91 75666 32455